

Project 07: RAG + MCP — Configuration Journey & Reasoning

This is a chronological, reasoned account of every configuration decision, repeated command, and troubleshooting juncture from `terraform init` through to working cross-provider MCP interoperability (Claude, Mistral). It focuses on *why* things were done, not just *what* — including places where things went wrong and why.

1. Terraform Bootstrapping

`terraform init`

Run once at the start, and again any time the backend or providers config changes. It:
Sets up the GCS backend (`rag-pipe-mcp-terraform-state` bucket, prefix `rag-mcp`) — pre-created manually before init, since Terraform can't create the very bucket it stores its own state in.
Downloads provider plugins (`hashicorp/google ~>5.0`, `hashicorp/random ~>3.6`).
Recurring issue: In Cloud Shell, `terraform init` kept showing the "install terraform" prompt even after installing via apt. Root cause: three `terraform` binaries existed on `PATH` (`/usr/bin`, `/bin`, `/google/bin`), and the shell had cached (hashed) a stale lookup from before install. Fixed with `hash -r`, which clears Bash's command-path cache — a good general lesson: if a freshly installed CLI tool "isn't found" despite being present, suspect shell caching before assuming installation failed.

`terraform plan` / `terraform apply` — run repeatedly

These aren't one-off commands; they were run after nearly every `.tf` file change, and multiple times per session due to transient failures. Reasons for repetition:

Propagation delay: Right after enabling a new API (e.g. `cloudresourcemanager.googleapis.com`), Google's systems take a minute or two to reflect the change. `terraform plan` would fail with `SERVICE_DISABLED` even though `gcloud services list --enabled` already showed it enabled. The fix wasn't a config change — just retrying after a short wait.

Flaky/transient network errors: A second wave of applies (from Cloud Shell, the next day) hit intermittent `storage.objects.get` errors and IPv6 DNS blips (`cannot assign requested address`) — different resources failed on each retry, with no pattern. This is normal transient cloud API flakiness, not a real problem; resolved by just retrying `terraform apply` until it succeeded.

The chicken-and-egg API problem: `cloudresourcemanager.googleapis.com` was never enabled and never added to `apis.tf`, because it's the API that lets Terraform manage the *enablement* of other APIs. You can't use Terraform to enable the one API Terraform itself depends on to manage state — it has to be enabled directly:

```
EO
SB
```

```
gcloud services enable cloudresourcemanager.googleapis.com
```

`terraform import`

Used once, after a real incident (see §3, "the race condition and state recovery" below) — to reconcile Terraform's state with a resource that already existed in GCP but wasn't tracked in state:

```
EO
SB
```

```
terraform import google_sql_database_instance.rag_pg rag-pipe-mcp/rag-pipe-mcp-pg
```

2. Architecture Decisions and Why

Private Cloud SQL, no public IP

`ipv4_enabled = false` in `database.tf`. This is the standard production pattern — a database should never be reachable from the open internet, only from within its own VPC. This single decision is the source of nearly every "why can't I connect from my laptop/Cloud Shell" problem later in the project (see §5).

IAM database authentication instead of password auth

`cloudsql.iam_authentication = "on"` DB flag, plus `google_sql_user` with `type = "CLOUD_IAM_SERVICE_ACCOUNT"`. Chosen over static passwords because:

Ties DB access to the service account's actual GCP identity — no secret to leak, rotate, or store.

Removed the need for `secrets.tf` (Secret Manager) entirely, once adopted — a password-based secret became unnecessary and was deleted.

Gotcha discovered: GCP requires the `.gserviceaccount.com` suffix stripped from the IAM DB username, and — critically — this stripping has to happen *twice*, independently, in two different places: once in the Terraform `google_sql_user.name` field, and once again in the Python connector code (`embed.py`'s `DB_USER`). These are two separate systems (Terraform's registration of the DB user vs. the Python client's connection string) that both need the same transformation applied.

Serverless VPC Access connector

Required because Cloud Run, by default, lives *outside* any VPC — even though your Cloud Run service and your private Cloud SQL instance are in the "same project," they don't automatically share network reachability. The VPC connector (`10.8.0.0/28`) is the bridge that lets Cloud Run reach the private IP.

Cloud Run `--allow-unauthenticated` + app-layer auth (planned)

External MCP-calling models (Claude, Mistral, etc.) can't present a GCP identity token — so IAM-based invoker restriction (`roles/run.invoker` scoped to specific principals) isn't usable here. Decision: leave Cloud Run's IAM layer open (`allUsers` granted `run.invoker`), and enforce access control at the *application layer* instead — inside `query_rag_pipeline`, via an API key or OAuth check. This is still a TODO/deferred item, but the architectural reasoning was locked in early: **IAM invoker and app-layer auth are two different tools for two different threat models** — IAM invoker is for "only my other GCP services can call this," app-layer auth is for "anyone on the internet can reach it, but they need a valid credential."

CI+CD combined in one `cloudbuild.yaml`

Departure from the pattern in other portfolio projects (04/05, 06), where Cloud Build only does CI (build → push → trigger a Vertex AI batch job). Here, Cloud Run is a **persistent service**, so the same pipeline also has to *deploy* — build → push → `gcloud run deploy`, all as one continuous pipeline. Cloud Deploy (GCP's managed, multi-environment progressive-rollout tool) was considered and explicitly rejected for this iteration: it's overkill for a single-environment POC, and its actual value proposition (staged rollout across dev/staging/prod with approval gates) doesn't apply until a future multi-env iteration.

Ingestion/embedding kept out of the live MCP tool

`ingest.py` / `chunk.py` / `embed.py` run as an occasional, one-off **Cloud Run Job**, never as part of the live `query_rag_pipeline` call path. Reasoning: if ingestion were wired into the MCP tool, every single query would risk re-scanning the bucket and re-embedding already-indexed documents — wasteful, slow, and prone to creating duplicate vector rows. The MCP tool should only ever do retrieval + generation, assuming the vector store is already current.

3. The Two-Pass Cloud Run Deploy (and the digest race condition)

Why a two-pass deploy was needed

`mcp_wrapper.py`'s `TransportSecuritySettings.allowed_hosts` needs the **real** Cloud Run service hostname to validate incoming request `Host` headers (DNS-rebinding protection). But that hostname is only assigned *after* Cloud Run creates the service for the first time — it's not knowable in advance. So:

First deploy: ship the image with a placeholder hostname in `allowed_hosts`.

Get the real URL: `gcloud run services describe ... --format="value(status.url)"` (or the `mcp_server_url` Terraform output).

Update `mcp_wrapper.py` with the real hostname.

Second deploy: rebuild, push, redeploy.

After that, every future deploy is single-pass — the hostname is stable across redeploys, only changing if the service were deleted and recreated.

The recurring "stale `:latest` digest" race condition

This bit multiple times across the project, always with the same symptom: after a rebuild+redeploy, `curl` tests kept showing *old* behavior (e.g. a 421 error that had supposedly already been fixed in the source code).

Root cause: Docker's `:latest` tag is just a mutable pointer, not a content guarantee. In an automated pipeline that does `docker build` → `docker push` → `gcloud run deploy --image=...:latest`, there's a real race: if the deploy step resolves the `:latest` tag *before* the push step finishes re-pointing it to the new digest, Cloud Run ends up running the **previous** image, even though the build "succeeded" and the tag now technically points at the new one.

How this was diagnosed, every time: never trust that a redeploy "worked" just because the CLI reported success. The reliable check is comparing two digests directly:

```
gcloud run revisions describe <REVISION_NAME> --region=europe-west4 --
format="value(spec.containers[0].image)"
gcloud artifacts docker images list europe-west4-docker.pkg.dev/rag-pipe-mcp/mcp-server-repo/mcp-server --include-tags
```

If the revision's running digest doesn't match the digest currently tagged `:latest` in Artifact Registry, the deploy pulled a stale image — full stop, no further debugging of "why doesn't my fix work" makes sense until this is confirmed to match.

The fix, every time it recurred: an explicit manual redeploy forces Cloud Run to re-resolve `:latest` fresh:

```
gcloud run deploy rag-pipe-mcp-mcp-server --image=europe-west4-docker.pkg.dev/rag-pipe-mcp/mcp-server-repo/mcp-server:latest --region=europe-west4
```

A related, subtler version of this same bug: at one point, `cloudbuild.yaml`'s explicit `docker push` step had gone missing (only 3 of the intended 4 steps were present — `find`, `build`, `deploy`, with `push` dropped). Without a push step, the deploy step was pulling whatever was *already* in Artifact Registry from a previous build — the newly-built image never made it there at all. This wasn't caught until the Cloud Build console log was inspected step-by-step and the missing step was noticed directly — a reminder that "the file I'm looking at locally" and "the file that actually built" can diverge if a commit didn't include what you think it did (see §7, "commit/build mismatches," below).

4. IAM Permission Chain — Why So Many Bindings Were Needed

Every time a new *actor* (a service account acting on your behalf) needed to do a new *thing*, a new IAM

binding was required. This wasn't over-engineering — each one corresponds to a genuinely distinct permission boundary:

Actor	Needs to...	Role granted
Compute default SA (<code>...-compute@...</code>)	Read its own Cloud Build staging bucket	<code>roles/storage.objectViewer</code>
Compute default SA	Push images during early builds	<code>roles/artifactregistry.writer</code>
<code>pipeline_sa</code>	Deploy to Cloud Run (once builds ran as <code>pipeline_sa</code> , not the default SA)	<code>roles/run.admin</code>
<code>pipeline_sa</code>	Be impersonated by Cloud Build's own identity	<code>roles/iam.serviceAccountUser</code> (self-granted)
<code>pipeline_sa</code>	Pull the image it's about to deploy	<code>roles/artifactregistry.reader</code>
<code>pipeline_sa</code>	Push the image at the final build step	<code>roles/artifactregistry.writer</code>
<code>pipeline_sa</code>	Log in to the Postgres instance via IAM	<code>roles/cloudsql.instanceUser</code> (DB-level, separate from <code>cloudsql.client</code>)
Your own Google account	Impersonate <code>pipeline_sa</code> locally, for one-off scripts	<code>roles/iam.serviceAccountTokenCreator</code>
Cloud Build's own runtime SA (<code>...@cloudbuild.gserviceaccount.com</code>)	Deploy to Cloud Run (once the <i>trigger</i> itself, not <code>pipeline_sa</code> , was found to be the actual build identity)	<code>roles/run.admin</code>
Cloud Build's runtime SA	Act as <code>pipeline_sa</code> when deploying the service to run as <code>pipeline_sa</code>	<code>roles/iam.serviceAccountUser</code> on <code>pipeline_sa</code>

Key conceptual distinction learned here: GCP IAM roles ("can this identity do X on GCP") and DB-level IAM authentication ("can this identity log into this specific Postgres instance") are two entirely separate

systems. Granting `cloudsql.client` alone was not enough — `cloudsql.instanceUser` (a different, DB-login-specific role) was also required, plus the DB flag, plus DB-side user registration via `google_sql_user`. All four had to be true simultaneously.

Impersonation, conceptually: you cannot "log in as" a service account — SAs have no interactive login of their own. Impersonation always requires a real, already-authenticated human identity behind it: you log in as yourself, then Google mints tokens as the target SA on your behalf, provided you've been granted `serviceAccountTokenCreator` on that SA. This is why `gcloud auth application-default login --impersonate-service-account=...` still opens a browser window asking you to sign in with your own Google account — that's the real identity being authenticated; the SA identity comes after, as a token swap.

5. Private-IP Reachability — The Recurring Wall

This was the single most repeated category of problem in the whole project, hit independently from three different environments:

Local laptop running `embed.py` directly: `CloudSQLIPITypeError` — the Cloud SQL Python Connector defaults to preferring a *public* IP unless told otherwise. Fixed with `ip_type="PRIVATE"` in the `connector.connect(...)` call.

Still from the laptop, even after that fix: no path to the private IP at all, since a laptop is never inside the VPC.

Cloud Shell, same script, same `ip_type="PRIVATE"` fix already applied: `TimeoutError: [Errno 110] Connection timed out`. This was the moment it became clear the fix wasn't the issue — Cloud Shell, despite feeling like it's "inside Google," is **not** inside your specific VPC. It's a separate, temporary environment with no peering to `rag-pipe-mcp-vpc`.

`gcloud sql connect` / **Cloud SQL Proxy from Cloud Shell**: same wall, different symptom (`dial tcp 10.200.0.3:3307: i/o timeout`), confirmed even with the `--private-ip` proxy flag.

The generalized lesson, stated explicitly once it became clear: private-IP-only Cloud SQL is unreachable from *anywhere* outside the VPC — not just your laptop, but also Cloud Shell, by design. The only legitimate paths in are: something already inside the VPC (Cloud Run + VPC connector, a Compute Engine VM, GKE), or a VPN/Interconnect bridging an external network into the VPC.

The resolution, reused for every subsequent private-IP-requiring task: Cloud Run Jobs. Since the MCP server (a Cloud Run *Service*) already proved the VPC connector path works, the same pattern was reused for one-off admin tasks by creating a Cloud Run *Job* that runs the **same container image**, just with a different command/args override:

```
gcloud run jobs execute rag-pipe-mcp-embed-job --region=europe-west4 --args=create_extension.py --update-env-vars=POSTGRES_PASSWORD=...
gcloud run jobs execute rag-pipe-mcp-embed-job --region=europe-west4 # normal embed.py run
```

This is the key reusable insight of the whole project: **one image, multiple entryptoints** — a persistent Cloud Run *Service* for the live MCP tool, and an on-demand Cloud Run *Job* (same image, different `command`/`args`) for anything that needs the same network access but only occasionally.

A note on `gcloud run jobs execute`'s actual flag support, discovered by trial and error: it does *not* support a `--command` override — only `--args` and `--update-env-vars`. An early attempt with `--command=python` failed with "unrecognized arguments." Since the job's container `command` (`["python"]`) was already fixed at the Terraform level, only the `args` (which script to run) needed overriding per-execution.

6. Postgres Superuser Operations — Why `postgres`, Not `pipeline_sa`

Two operations turned out to be impossible for `pipeline_sa` (the IAM-authenticated DB user) no matter what GCP IAM roles it held:

`CREATE EXTENSION IF NOT EXISTS vector;` — creating a Postgres extension requires actual Postgres superuser privilege, which is a database-internal concept, not something any GCP IAM role can grant.

`GRANT ALL ON SCHEMA public TO "...";` — schema-level privilege grants likewise require superuser.

Resolution: reset the built-in `postgres` user's password (`gcloud sql users set-password postgres --instance=... --password=...`), then connect as `postgres` (password auth, not IAM) specifically to run these two one-time commands. Since `postgres` still needs to reach the private IP, this had to run through the same Cloud Run Job mechanism as everything else in §5 — a small script (`create_extension.py`) using the same `cloud-sql-python-connector` pattern as `embed.py`, but with `user="postgres"`, `password=<the reset password>`, `enable_iam_auth=False`.

Order of operations mattered: the extension had to be created *before* the schema grant could matter, and both had to happen before `embed.py`'s first successful run — `embed.py` failed twice in sequence, first with `permission denied to create extension "vector"`, then (after the extension was created) with `permission denied for schema public`, since creating the extension didn't implicitly grant `pipeline_sa` any schema-level write access.

7. Commit/Build Mismatches — A Recurring Debugging Trap

Multiple times, a fix was written, verified locally, and pasted for review — correct in every case — yet the *deployed* container kept exhibiting the *old*, pre-fix bug. Each time, the apparent paradox had a mundane cause:

File genuinely not pushed yet: a fix existed locally but hadn't been committed/pushed before triggering a job execution or build.

Stale image digest (§3): the fix was pushed and built, but the deploy step pulled an old `:latest` due to the tag-race condition.

Wrong commit built: Cloud Build's log explicitly shows which commit SHA it checked out (`HEAD is now at <sha> <commit message>`) — at one point, a build ran against an *older* commit (`6c94d31`) that predated a newer, already-pushed fix (`842ba03`), simply because that older commit was the one Cloud Build had queued/processed at that moment, not because anything was actually broken.

Missing pipeline step: the `docker push` step had silently gone missing from `cloudbuild.yaml`'s active (uncommented) block at some point, so builds "succeeded" but never actually updated the image Cloud Run would later pull.

The diagnostic discipline that resolved all of these: never trust "I pasted the fix" as proof it's live. Instead:

`git log --oneline -5` to see the actual commit history and confirm which commit is `HEAD`.

`git show HEAD:"path/to/file.py"` to view the *actually committed* content of a file, independent of what your editor shows — this caught a case where the file editor view and the git-tracked content had silently diverged.

Cross-reference the Cloud Build console's `GitCommit:` line against `git log` to confirm the build ran against the commit you think it did.

Cross-reference the deployed revision's image digest against the newest digest in Artifact Registry (§3) to confirm the *build* actually reached the *runtime*.

8. Middleware Debugging — Iterating Through Starlette API

Mismatches

Adding a compatibility shim (see §9 for *why* it was needed) required three separate rounds of runtime failures, each a genuinely different Starlette API misunderstanding, each only visible from the **actual container startup logs** (not from local code review, since the bug was version/API-specific and the code “looked” reasonable each time):

First attempt: `app.middleware("http")(liveness_probe_middleware)` — this is Starlette's older decorator-only pattern; it's not a directly-callable method on an already-constructed `Starlette` object. Failed with `AttributeError: 'Starlette' object has no attribute 'middleware'. Did you mean: 'add_middleware'?` — the error message itself pointed at the fix.

Second attempt: wrapping `mcp.streamable_http_app()`'s output inside a new outer `Starlette` app via `Mount("/", app=mcp_app)`, with middleware attached to the *outer* app. This built and deployed, but the container never started — Cloud Run's health check timed out with no Python traceback at all, which strongly suggested the outer wrapping broke ASGI lifespan-event forwarding to the inner MCP session manager (the inner app's startup/shutdown events never got a chance to run). Abandoned in favor of attaching middleware directly to the app `streamable_http_app()` already returns, rather than nesting it inside another app.

Third attempt: `app.add_middleware(BaseHTTPMiddleware, dispatch=LivenessProbeMiddleware().dispatch)` — this pre-instantiates `LivenessProbeMiddleware()` with no arguments, but `BaseHTTPMiddleware.__init__` requires an `app` argument (the ASGI app it wraps) that isn't available yet at that point in the code. Failed with `TypeError: BaseHTTPMiddleware.__init__() missing 1 required positional argument: 'app'`.

Working version: `app.add_middleware(LivenessProbeMiddleware)` — passing the *class*, not an instance. This is the standard Starlette pattern: `add_middleware` handles constructing the middleware class with the correct `app` argument internally, at the point the middleware stack is actually built.

Why each of these only surfaced at runtime, not at code-review time: all four versions are syntactically valid Python that would pass a casual read-through; the errors are specific to Starlette's internal middleware-construction API, which isn't something general Python knowledge predicts correctly. This is the concrete argument for why “the logs are the source of truth, not the pasted code” was the repeated diagnostic move throughout this whole debugging arc.

9. Divergent Philosophies: MCP Spec vs. Mistral's Connector Validator

This was the most conceptually interesting juncture in the project — a case where **two correct, standards-referencing implementations genuinely disagreed** on what a bare, sessionless GET request to an MCP endpoint should mean.

The MCP Streamable HTTP spec's position

Per the official 2025-03-26 Streamable HTTP transport spec, a GET request without an established session is a **protocol error** — the server should reject it, because a GET without context isn't a meaningful operation in the session-based model Streamable HTTP is built around. Your server, using Anthropic's official Python MCP SDK, correctly implemented this: a bare GET returned `400 Bad Request: Missing session ID`. This is not a bug — it's textbook-correct behavior per the spec your server actually speaks.

Mistral's Connector registration validator's position

Mistral's documented connector health-check logic (confirmed via their own help article, quoted verbatim in their troubleshooting guide) expects something different and simpler:

No authentication required → the endpoint should return `200` on a first, bare GET.

OAuth required → it should return `401` with a `WWW-Authenticate` header.

Neither of these matches "400, because you sent a GET without a session" — Mistral's validator isn't speaking the MCP session protocol at that point; it's doing a simpler, REST-style liveness/auth-detection probe *before* it ever gets to real MCP traffic. This is arguably a step *outside* the MCP spec entirely — a Mistral-specific compatibility convention layered on top.

Why this mismatch was hard to diagnose

The failure mode from Mistral's side was an opaque `400 Bad Request: server_unreachable` at connector-creation time — no indication that the actual issue was a philosophical disagreement about GET semantics. This required:

Reading Mistral's own documented troubleshooting checklist (found via search, since the "official" URL initially given turned out to be dead/broken — a real dead-end that had to be backtracked from honestly rather than asserted as correct).

Recognizing that the checklist's expected response codes (`200`/`401`) didn't match your server's actual, spec-correct behavior (`400`).

Concluding that satisfying Mistral required **adding a deliberate special case** — not fixing a bug, but building a compatibility shim for a specific external validator's non-standard expectation.

The resolution: a scoped compatibility shim, not a spec violation

Rather than changing the *real* session-handling logic (which remains spec-correct for all genuine MCP traffic), a middleware was added that intercepts *only* the exact case Mistral's probe uses — a bare GET on `/mcp` with no `mcp-session-id` header — and returns `200` directly, before the request ever reaches the real MCP session logic. Every other request (real POST-based MCP traffic, GETs *with* a session ID) is untouched and still spec-correct. This is a deliberate, narrowly-scoped exception, not a redefinition of the server's actual protocol behavior — the distinction mattered enough to document explicitly in the middleware's own docstring.

Broader lesson

Two different LLM providers' MCP integrations (Anthropic's own SDK/spec vs. Mistral's Connector product) can have genuinely different, non-overlapping assumptions about the same protocol, even though both claim MCP compatibility. "Speaks MCP" doesn't guarantee identical liveness-check conventions across providers — each provider's *specific* validator may impose its own additional, undocumented-in-the-core-spec requirements. This is exactly the kind of interoperability friction the MCP standard is meant to reduce over time, but hadn't fully ironed out as of this build.

10. Final Working State — What "Done" Looked Like

Server: `mcp_wrapper.py`, deployed via Cloud Run Service, using the official Anthropic MCP Python SDK v2 (`MCPServer`, `streamable_http_app`), with `TransportSecuritySettings` locked to the real production hostname(s), plus a scoped middleware for Mistral's liveness-check convention.

Data path: private Cloud SQL (IAM-authenticated) ← pgvector ← `embed.py` (Cloud Run Job) ← `chunk.py` ← `ingest.py` ← GCS bucket of source PDFs.

CI/CD: single `cloudbuild.yaml`, four steps (debug `find`, `docker build`, `docker push`, `gcloud run deploy`), triggered automatically on push to the `rag-pipe` branch.

Verified interoperability: two independent, non-Anthropic-affiliated model providers (Claude via its native `mcp_servers` API parameter, and Mistral via its Connectors API) both successfully connected to

the deployed endpoint and correctly invoked `query_rag_pipeline`, receiving grounded, source-attributed answers back.

Deferred, explicitly tracked: API key/OAuth gate inside `query_rag_pipeline` itself (the endpoint is currently open to any caller who knows the URL); postprocessing (safety filter, custom checks, RAGAS evaluation); final decision on `embed.py`'s vector-store library (generic `langchain_postgres` vs. Google-native `langchain-google-cloud-sql-pg`).